



Experiment-9

Student Name: Pranav Chauhan

Branch: BE CSE (cyber security)

Semester: 5th

Subject Name: Computer Networks

UID: 24BCY70004

Section/Group: 24AIT_KRG-G2

Date: 16/09/2026

Subject Code: 24CSH-337

1. Aim/Overview of the practical:

To implement **UDP socket communication using Python** by creating a client-server application that sends and receives messages using **UDP protocol**.

2. Software/Tools Required:

- Python 3.x
- PyCharm / VS Code
- Python socket module
- Windows/Linux operating system

3. Theory:

A) UDP Socket

UDP (User Datagram Protocol) is a connectionless communication protocol used for sending and receiving data between applications. Unlike TCP, UDP does not establish a dedicated connection before communication.

B) UDP Socket Programming

Python provides the socket module for network communication. A UDP socket is created using `socket.AF_INET` for IPv4 and `socket.SOCK_DGRAM` for UDP communication.

C) Client-Server Communication

In UDP communication, the server binds its socket to an IP address and port number and waits for messages. The client sends messages directly to the server address without establishing a connection. In this experiment, 127.0.0.1 and port 5000 are used.

D) Sending and Receiving Data

The client uses `sendto()` to send messages to the server. The server uses `recvfrom()` to receive the message along with the client's address. The server can then send a reply using `sendto()`.

E) Connection Termination

The communication continues until the message "Over" is entered. After receiving this message, the program exits the communication loop and closes the UDP socket.

4. Steps for Experiment/Practical:

- 1) Import the socket module in both `server.py` and `client.py`.
- 2) Create a UDP socket using `socket.AF_INET` and `socket.SOCK_DGRAM`.
- 3) In `server.py`, bind the socket to `127.0.0.1` and port `5000`.
- 4) Start the UDP server and make it wait for incoming messages.
- 5) In `client.py`, define the server address as `127.0.0.1` and port `5000`.
- 6) Start the UDP client.
- 7) Enter a message in the client and send it using `sendto()`.
- 8) Receive the message on the server using `recvfrom()`.
- 9) Decode and display the received message.
- 10) Enter a reply on the server and send it back using `sendto()`.
- 11) Receive and display the server's reply on the client.
- 12) Continue the communication by sending and receiving messages.
- 13) Enter "Over" to terminate the communication.
- 14) Close the UDP sockets on both the client and server.



5. Output:

```
server.py X client.py X
server.py > ...
1 import socket
2
3 ## Creating a UDP Socket
4 server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5
6 ## A socket is a software endpoint through which one program sends or receives data
7
8 server_socket.bind(("127.0.0.1", 5000))
9
10 print("UDP Server started")
11 print("Waiting for messages...")
12
13 while True:
14     message, client_address = server_socket.recvfrom(1024)
15     message = message.decode()
16     print("Client:", message)
17     if message == "Over":
18         break
19     reply = input("Enter reply: ")
20     server_socket.sendto(reply.encode(), client_address)
21
22 server_socket.close()
23
24 print("Server closed")
```

```
server.py X client.py X
client.py > ...
1 import socket
2
3 client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4
5 server_address = ("127.0.0.1", 5000)
6
7 print("UDP Client started")
8
9 while True:
10     message = input("Enter message: ")
11     client_socket.sendto(message.encode(), server_address)
12     if message == "Over":
13         break
14     reply, server_address_received = client_socket.recvfrom(1024)
15     print("Server:", reply.decode())
16
17 client_socket.close()
18
19 print("Client closed")
```

Problems Output Debug Console Terminal Ports Python: client + -

```
PS C:\PranavData\main data\CU\notes\5th sem\cn\Practicals\exp-9\UDP_Socket_Experiment> & C:\Users\Pranav\AppData\Local\Programs\Python\Python314\python.exe "c:/PranavData/main data/CU/notes/5th sem/cn/Practicals/exp-9/UDP_Socket_Experiment/server.py"
UDP Server started
Waiting for messages...
Client: hi
Enter reply: hello

```

```
PS C:\PranavData\main data\CU\notes\5th sem\cn\Practicals\exp-9\UDP_Socket_Experiment> & C:\Users\Pranav\AppData\Local\Programs\Python\Python314\python.exe "c:/PranavData/main data/CU/notes/5th sem/cn/Practicals/exp-9/UDP_Socket_Experiment/client.py"
UDP Client started
Enter message: hi
Server: hello
Enter message: 

```



6. Result

The UDP client-server application was successfully implemented using Python sockets. The client and server successfully exchanged messages using UDP, and the communication was terminated after sending "**Over**".

7. Learning Outcomes

- Understand the basics of **UDP socket communication**.
- Learn to create a UDP socket using Python.
- Understand the **client-server communication model**.
- Learn to send and receive messages using `sendto()` and `recvfrom()`.
- Understand how UDP communication is established and terminated.